

Problem Brief — Claims Exception Triage Assistant

Shilpi Verma · UHC Tech AI Transformation, Cohort 5 · Track 1 · August 2026

The problem

When a claim fails automatic processing it lands in an exception queue, and the analyst's first job is not fixing it — it is **working out why it is stuck**. The system's own "pend code" is often generic or stale (about 30% of our modelled queue carry codes that say nothing useful), so diagnosis means reading a free-text note and then checking three other systems by hand: prior authorisations, the claim's history, and the contract rate table. Worse, queues are worked oldest-first rather than worst-first. A \$42,000 surgical claim that is already past its deadline can sit behind a \$45 office visit that simply arrived earlier.

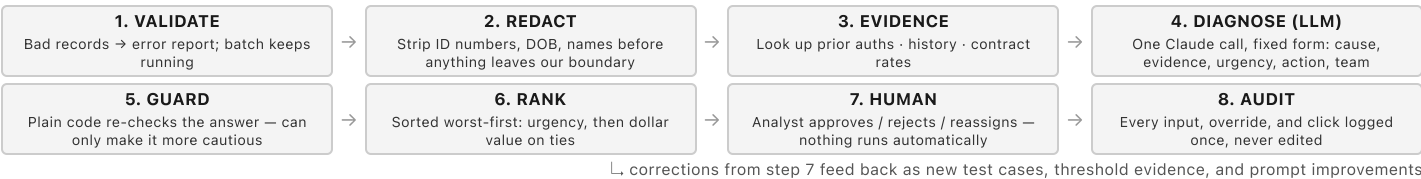
Target user: the claims operations analyst working that queue.

The pain: most of the handling time is diagnosis, and the order of work ignores impact.

The solution

An assistant that pre-diagnoses every stuck claim — the likely cause, a plain-English summary quoting the evidence it used, an urgency score, a suggested next action, and which team owns it — and then puts a human in front of every decision. The analyst starts at "verify" rather than "investigate", and the queue is ordered by impact instead of arrival date.

Architecture — how a claim flows through it



Trust design — the model proposes, code disposes

- **The rules can only escalate.** Past deadline or over \$10,000 forces urgency up. A test that fails the build proves the rules can never lower urgency.
- **The team is derived, never guessed.** Each of the 7 causes maps to exactly one team, so an inconsistent cause-and-team pairing cannot be recorded.
- **Uncertainty goes to a person.** Below 65% confidence, or when the stated cause is contradicted by a source system, the claim routes to a senior analyst. The model is structurally prevented from choosing that route itself, so it cannot hedge its way out of a review.
- **Every action needs a human.** The system executes nothing.

Measured results — live Claude calls

Sealed set — 25 claims held back and scored exactly once, after the prompt was frozen:

Metric	Result
Correct cause identified	96% (24 of 25)
Routed to the correct team	96%
Most serious claims surfaced in the top 10	100%
Urgency order vs. true severity	0.83 (1.0 = perfect agreement)

Development set — 50 claims, against a deliberately simple alternative:

Metric	Rule-based baseline	This system
Correct cause identified	72%	100%
Routed to the correct team	76%	100%
Most serious claims in the top 10	50%	100%
Urgency order vs. true severity	0.55	0.82

The baseline is the honest "why an LLM at all?" comparison: guess the cause from the pend code, rank by dollar value alone. Where pend codes are generic it guesses blindly — precisely where analysts have to think.

Two caveats we found ourselves, stated rather than buried:

The development set is saturated. Prompt versions V1, V2 and V3 all score identically on it, so it can no longer distinguish prompt quality. The same 50 claims hold the baseline to 72%, so the set is not trivially easy — it simply no longer discriminates at this model tier. That is why the sealed number above is the one to trust.

The single sealed-set miss was our fault twice over. The note read "member shows termed coverage" — unambiguously an eligibility problem — but no contract rate appeared in the table, and our prompt tells the model a missing rate is positive evidence for a pricing problem. It followed our rule over a clear note, at 85% confidence.

Tracing it found the deeper fault. The rate table is keyed by provider-and-procedure, which several claims **share**. Our fix for an earlier bug strips that key for every pricing claim — and because the key is shared, it also stripped rates for other claims that legitimately had one. **15 of 75 claims (20%) are shown a missing rate they should not be.** So the model was fed corrupted evidence and told to trust it absolutely. This is the earlier bug recurring in mirror image: we fixed "shared keys wrongly fill a gap" and created "shared keys wrongly empty one," because we never checked the reverse direction.

We did not fix either fault before reporting, because doing so would mean re-running the sealed set and a held-out set scored twice is not held out. Both are logged for the next version. The honest lesson: no confidence threshold would have caught this — the model was not uncertain, it was misled — which is precisely why the rules layer and the human gate exist.

Learning from analysts

Every correction an analyst makes is captured, with the cause they believe is correct. Corrections become three things: permanent regression tests, evidence for tuning the confidence threshold, and candidate examples for the next prompt version.

One rule governs it: **an approval is not a label**. A rejection costs the analyst effort, so it carries information; an approval may equally mean someone clicked through quickly. Treating approvals as confirmed-correct would feed the system its own output and let mistakes reinforce themselves. Nothing retrains automatically — every change passes through a human and a versioned release.

Scope

Built: synthetic queue of 75 claims; per-claim diagnosis; the rules layer; ranked review interface; evaluation harness with baseline comparison; audit log; analyst feedback loop. 43 automated tests.

Deliberately excluded: real payment or denial decisions, live system integration, automatic execution of any action, model fine-tuning, and multi-agent architecture — each with reasoning recorded in the PRD.

Honest constraints: every number here comes from synthetic data whose difficulty we controlled, so the time-saving case remains a hypothesis until it is tried alongside real analysts on real claims. Production would need the model hosted inside the enterprise's own boundary under a data agreement, sending it only the specific fields it needs — at which point the redaction layer becomes a second line of defence rather than the primary control.